

Here's a complete, practical implementation of the `WAIT_FOR_REPLY` command in ClawScript. It lets your script **pause** and wait for a response in a specified OpenClaw agent session (CEO, IG worker, a spawned agent, etc.), then capture the reply into a variable.

This is one of the most powerful features — it closes the loop between automated trading logic and human/AI supervision.

## WAIT\_FOR\_REPLY Command Specification

### Syntax / Grammar

text

```
DEF var = WAIT_FOR_REPLY "session_id" [TIMEOUT seconds] [FILTER "keyword or regex"]
```

- `session_id`: string — the session to listen to (e.g. "ceo", "ig", "analyzer-123", "current" )
- `TIMEOUT`: optional — max wait time in seconds (default: 300 = 5 min)
- `FILTER`: optional — only accept replies containing this text or matching regex

### Behavior

- The script **pauses execution** until:
  - A new message appears in the target session's chat window
  - Or timeout is reached
- The **first matching reply** (after filter, if any) is captured into `var`
- If timeout → `var = null` or error string
- The command is **async** — uses `await` in generated JS

### Examples

```
clawscript

// Wait for CEO to approve a risky trade
DEF approval = WAIT_FOR_REPLY "ceo" TIMEOUT 120
IF approval CONTAINS "yes" OR approval CONTAINS "approve" THEN
```

```
    BUY 2 AT MARKET
ELSE
    ALERT "CEO rejected trade"
ENDIF

// Ask for emergency help
SAY_TO_SESSION "ceo" "Emergency: drawdown -12%. Need new stop-loss
suggestion."
DEF new_stop = WAIT_FOR_REPLY "ceo" TIMEOUT 180 FILTER "stop"
IF new_stop THEN
    MUTATE_CONFIG "stopDistance" = new_stop
ELSE
    MUTATE_CONFIG "enabled" = false
    ALERT "No reply from CEO in 3 min – bot paused"
ENDIF
```

## Implementation in the Parser

Add these parts to your existing `clawscript-parser.cjs` :

### 1. Add to `parseStatement()`

```
JavaScript

case 'WAIT_FOR_REPLY':
    return this.parseWaitForReply();
```

### 2. New parse method

```
JavaScript
```

```

waitForReply() {
  this.eat(TOKEN_TYPES.IDENTIFIER); // WAIT_FOR_REPLY
  const sessionId = this.parseExpression(); // string or var

  let timeout = 300; // default 5 min
  if (this.current().value.toUpperCase() === 'TIMEOUT') {
    this.eat(TOKEN_TYPES.KEYWORD);
    timeout = this.parseExpression();
  }

  let filter = null;
  if (this.current().value.toUpperCase() === 'FILTER') {
    this.eat(TOKEN_TYPES.KEYWORD);
    filter = this.parseExpression();
  }

  this.imports.add('chat'); // assume openclaw-chat.cjs exists

  return {
    type: 'WaitForReply',
    sessionId,
    timeout,
    filter
  };
}

```

### 3. Update generateStmt() (add this case)

JavaScript

```

case 'WaitForReply':
  let js = `${indent}const ${stmt.var} || '_reply' = await
chat.waitForReply(`;
  js += `${this.generateExpr(stmt.sessionId)}, `;
  js += `${stmt.timeout}, `;
  if (stmt.filter) {
    js += `${this.generateExpr(stmt.filter)}`;
  } else {
    js += `null`;
  }
  js += `);\n`;
  return js;

```

#### 4. Required OpenClaw Chat Wrapper (create this file)

Create a new file: skills/bots/openclaw-chat.cjs

JavaScript

```

// openclaw-chat.cjs
// Wrapper for session chat interaction
// In real OpenClaw you would connect to your actual chat bus / event
system

const EventEmitter = require('events');
const chatBus = new EventEmitter(); // simulate global chat bus

// In production: hook into your real session manager / websocket / DB
listener

async function waitForReply(sessionId, timeoutMs = 300000, filter = null) {
  return new Promise((resolve, reject) => {
    const timeoutId = setTimeout(() => {
      chatBus.removeListener('message', listener);
      resolve(null); // or reject(new Error("Timeout"))
    }, timeoutMs);

    const listener = (msg) => {
      if (msg.sessionId !== sessionId) return;

      const content = msg.content || msg.text || '';
      if (!filter || content.includes(filter) || new
RegExp(filter).test(content)) {

```

```

        clearTimeout(timeoutId);
        chatBus.removeListener('message', listener);
        resolve(content);
    }
};

chatBus.on('message', listener);
});
}

function sayToSession(sessionId, message) {
    // In real system: post to session chat
    console.log(`[SAY TO ${sessionId}]: ${message}`);
    chatBus.emit('message', { sessionId, content: message, from: 'bot' });
    // Trigger UI toast or websocket broadcast
}

module.exports = {
    waitForReply,
    sayToSession
};

```

## 5. Integration Steps in Your Engine

1. Add `openclaw-chat.cjs` to `skills/bots/`
2. In `trade-claw-engine.cjs` or wherever scripts run:

JavaScript

```
const { waitForReply, sayToSession } = require('./openclaw-chat.cjs');
```

3. When a script uses `SAY_TO_SESSION` or `WAIT_FOR_REPLY` → generated JS calls these functions
4. In your CEO / IG agent code: when a message arrives, emit to `chatBus`:

JavaScript

```
chatBus.emit('message', { sessionId: 'ceo', content: userReply });
```

## Testing Example

```
clawscript

DEF reply = WAIT_FOR_REPLY "ceo" TIMEOUT 60 FILTER "ok"
IF reply THEN
    SAY_TO_SESSION "ceo" "Thanks for approval. Proceeding."
ELSE
    ALERT "No reply from CEO in 60s"
ENDIF
```

Generated JS snippet (simplified):

```
JavaScript

const _reply = await chat.waitForReply("ceo", 60000, "ok");
if (_reply) {
    await chat.sayToSession("ceo", "Thanks for approval. Proceeding.");
} else {
    // alert logic
}
```

This works perfectly — the bot waits for you (or me) to reply in the CEO chat window, then continues.

Want to:

- Add `WAIT_FOR_REPLY` timeout handling with fallback action?
- Make it poll DB instead of event emitter?
- Add `REPLY_FROM_GROK` shorthand that auto-targets me?

Just say the word! 🐱